

Text Preprocessing for MCMF Program Descriptions

Text data is everywhere in data science—whether as unstructured text files or as text columns within structured datasets. Developing strong skills for cleaning, processing, and analyzing text is essential for extracting insights and building effective models.

Learning Objectives:

- Understand the importance of text preprocessing in NLP and data science workflows
- Learn how to clean and standardize raw text data
- Engineer features from text for downstream analysis and modeling
- Apply practical techniques to real-world datasets

Dataset Case Study: My CHI My Future (MCMF) Programs in Chicago

This case study explores real-world data from the My CHI My Future (MCMF) youth programs in Chicago. The dataset was selected for the [MCDC](#) practicum course (Spring 2023, Dr. Lizhen Shi) in partnership with [My CHI My Future](#) and community leader Stender Von Oehsen. The project aimed to address real community needs through data science, with results featured on the [MCDC Past Projects website](#).

Context:

- The MCMF initiative, led by the City of Chicago, connects youth to thousands of out-of-school programs, events, and resources across the city.
- The dataset includes detailed information about program names, descriptions, categories, dates, and locations, providing a rich resource for text analysis and feature engineering.
- Students worked with community partners to clean, analyze, and model this data, supporting efforts to improve youth engagement and access to opportunities.

This notebook demonstrates how to preprocess and analyze text data from a real civic dataset, preparing it for downstream modeling and actionable insights. This lesson was co-developed by **Dr. Lizhen Shi** and **Dr. Arend Kuyper** for the MCDC pathway courses (Summer 2025).

Notebook overview

This notebook walks through a full text-cleaning pipeline for the My CHI My Future programs data so learners can see the why behind each step. We'll: 1) Load the raw CSV and inspect structure/data quality. 2) Engineer date features (start year, duration) and prune to realistic records. 3) Focus on post-pandemic programs (2023-2025) and remove duration outliers. 4) Prepare category and text fields for downstream modeling. 5) Export a cleaned CSV used by later notebooks.

```
import pandas as pd
import numpy as np
```

Loading the data

The dataset is publicly available on the [Chicago Data Portal](#). For this lesson, we dropped the columns that aren't relevant to text analysis and kept only the text-focused fields (e.g., **program name** and **program description**) to simplify downstream cleaning and modeling.

Let's next load the dataset into a pandas DataFrame.

How we'll load it

- Source: `datasets/My_CHI._My_Future._Programs_20250826.csv` (export from the Chicago Data Portal).
- Keep text-focused columns; numeric and geo fields are out-of-scope for this lesson.
- If you're low on memory, sample a subset first (e.g., `df.sample(5000, random_state=42)`) to iterate quickly.

```
df = pd.read_csv('datasets/My_CHI._My_Future._Programs_20250826.csv', low_memory=False)
df.head()
```

	Program ID	Program Name	Description
0	138124	How to Finally Get Consistent with Healthy Eating	During this workshop you'll identify
1	138610	Book Zoomies Activity Sheets	These activity sheets from Book Zoom
2	137709	Book Zoomies Activity Sheets	These activity sheets from Book Zoom
3	138702	Dark Winds, Prey, Trickster and Reservation Do...	In honor of Native American Heritage

	Program ID	Program Name	Description
4	121785	Homework Help: Virtual Teacher in the Library	Get homework help from a certi

Now let's examine the structure of our dataset to understand what columns we have available and their data types. This will help us identify the text fields we can analyze and any data quality issues we need to address.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 460531 entries, 0 to 460530
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Program ID      460529 non-null  object
1   Program Name    460529 non-null  object
2   Description     460529 non-null  object
3   Category Name   460526 non-null  object
4   Start Date      460527 non-null  object
5   End Date        460527 non-null  object
6   Start Time      418345 non-null  object
7   End Time        418299 non-null  object
dtypes: object(8)
memory usage: 28.1+ MB
```

What to look for in info()

- Text fields: confirm they are non-null strings and spot obvious missing descriptions.
- Date fields: ensure they come in as `object` so we can convert to datetime next.
- Row count: note the baseline size before any filtering so you can track drops.

Data Cleaning

Engineer date features (start year, duration) and prune to realistic records.

Next, we'll create additional columns from the date fields to support our analysis. We'll extract the year from the start date and calculate the duration of each program in days. This will help us understand temporal patterns and program length characteristics.

```

# Create new columns: Start Year and Duration
print("\nCREATING NEW DATE-BASED COLUMNS")
print("="*40)

# Convert date columns to datetime format
print("Converting date columns to datetime format...")
df['Start Date'] = pd.to_datetime(df['Start Date'], errors='coerce')
df['End Date'] = pd.to_datetime(df['End Date'], errors='coerce')

# Extract year from start date
df['Start_Year'] = df['Start Date'].dt.year
# Convert to nullable integer type to handle NaN values properly
df['Start_Year'] = df['Start_Year'].astype('Int64')
print(f" Created 'Start_Year' column")

# Calculate duration between start and end dates (in days)
df['Duration_Days'] = (df['End Date'] - df['Start Date']).dt.days
print(f" Created 'Duration_Days' column")

# Show sample of new columns
print(f"\nSample of new columns:")
df.head()

```

CREATING NEW DATE-BASED COLUMNS

=====

Converting date columns to datetime format...

Created 'Start_Year' column

Created 'Duration_Days' column

Sample of new columns:

	Program ID	Program Name	Description
0	138124	How to Finally Get Consistent with Healthy Eating	During this workshop you'll identify
1	138610	Book Zoomies Activity Sheets	These activity sheets from Book Zoom
2	137709	Book Zoomies Activity Sheets	These activity sheets from Book Zoom
3	138702	Dark Winds, Prey, Trickster and Reservation Do...	In honor of Native American Heritage
4	121785	Homework Help: Virtual Teacher in the Library	Get homework help from a certi

Let's examine the distribution of programs across different years and analyze the quality of

our date data. This will help us understand temporal patterns and identify any data quality issues that need to be addressed.

Before filtering, we profile the new **Start_Year** and **Duration_Days** columns to see how reliable the dates are and where obvious issues (missing/negative/too long) show up.

```
# Display statistics about the new columns
print(f"\nStart Year distribution:")
year_counts = df['Start_Year'].value_counts().sort_index()
print(year_counts.head(10))

print(f"\nDuration statistics:")
duration_stats = df['Duration_Days'].describe()
print(duration_stats)

# Check for any issues with the data
print(f"\nData quality check:")
print(f"- Programs with missing Start Date: {df['Start Date'].isna().sum():,}")
print(f"- Programs with missing End Date: {df['End Date'].isna().sum():,}")
print(f"- Programs with negative duration: {(df['Duration_Days'] < 0).sum():,}")
print(f"- Programs with very long duration (>365 days): {(df['Duration_Days'] > 365).sum():,}")
```

```
Start Year distribution:
Start_Year
2020      208
2021    46987
2022    93328
2023   102343
2024   136620
2025    81024
2026      12
2054       3
Name: count, dtype: Int64
```

```
Duration statistics:
count    460525.000000
mean         34.023378
std        137.853833
min       -10577.000000
25%           0.000000
50%           0.000000
75%          63.000000
```

```
max          10227.000000
Name: Duration_Days, dtype: float64
```

Data quality check:

- Programs with missing Start Date: 6
- Programs with missing End Date: 6
- Programs with negative duration: 3
- Programs with very long duration (>365 days): 875

Data Cleaning: Focus on post-pandemic programs (2023-2025) and remove duration outliers.

Post-pandemic Programs

Based on our year distribution analysis, we can see that the program counts for 2020-2022 were significantly affected by the COVID-19 pandemic. From 2023 onwards, youth programs appear to have returned to normal levels.

For our text analysis, we'll focus on programs from **2023-2025** as these represent: - Post-pandemic program offerings that reflect current trends - More recent and relevant program descriptions - Data that is most applicable to current programming decisions

We'll also remove any programs with start years beyond 2025, as these are likely data entry errors given our current date.

```
# Filter the dataset to include only programs from 2023-2025
print("FILTERING DATASET BY YEAR")
print("="*30)

# Show original dataset size
print(f"Original dataset size: {len(df):,} programs")

# Filter for programs starting in 2023, 2024, or 2025
df_filtered = df[df['Start_Year'].isin([2023, 2024, 2025])].copy()

print(f"Filtered dataset size: {len(df_filtered):,} programs")
print(f"Programs removed: {len(df) - len(df_filtered):,}")
print(f"Percentage retained: {(len(df_filtered) / len(df) * 100):.1f}%")

# Show year distribution in filtered dataset
print(f"\nYear distribution in filtered dataset:")
filtered_year_counts = df_filtered['Start_Year'].value_counts().sort_index()
for year, count in filtered_year_counts.items():
```

```
percentage = (count / len(df_filtered)) * 100
print(f" {year}: {count:},} programs ({percentage:.1f}%)"
```

FILTERING DATASET BY YEAR

=====

Original dataset size: 460,531 programs

Filtered dataset size: 319,987 programs

Programs removed: 140,544

Percentage retained: 69.5%

Year distribution in filtered dataset:

2023: 102,343 programs (32.0%)

2024: 136,620 programs (42.7%)

2025: 81,024 programs (25.3%)

Checkpoint: post-year filter

Capture the retained row count and year distribution shown above; you can log this number in reports so downstream notebooks know the cohort size they are using.

Remove Duration Outliers

Now let's clean up the data based on program duration. We'll remove programs with unrealistic durations that could indicate data quality issues: - Programs with negative durations (end date before start date) - Programs with extremely long durations (over 1 year) which may be data entry errors - Zero duration programs represent single-day programs or events that last only a couple hours within the same day

Let's create a `Duration_Group` column to better categorize programs:

- Same day (0 days) - for events/programs within a single day
- 1 day - for programs spanning exactly one day
- 2-7 days - for short multi-day programs
- 1-4 weeks, 1-3 months, 3-6 months, 6-12 months - for longer programs

This will help ensure our text analysis is based on high-quality, realistic program data.

```
# Clean up data based on Duration_Days
print("DURATION-BASED DATA CLEANUP")
print("="*35)

print(f"Before duration cleanup: {len(df_filtered):},} programs")
```

```

# Check current duration issues
print(f"\nDuration quality issues in filtered data:")
negative_duration = (df_filtered['Duration_Days'] < 0).sum()
very_long_duration = (df_filtered['Duration_Days'] > 365).sum()
zero_duration = (df_filtered['Duration_Days'] == 0).sum()
missing_duration = df_filtered['Duration_Days'].isna().sum()

print(f"- Negative duration: {negative_duration:,} programs")
print(f"- Very long duration (>365 days): {very_long_duration:,} programs")
print(f"- Zero duration (same day programs): {zero_duration:,} programs")
print(f"- Missing duration: {missing_duration:,} programs")

# Apply duration-based filters
print(f"\nApplying duration filters...")

# Remove programs with negative durations
df_filtered = df_filtered[df_filtered['Duration_Days'] >= 0]
print(f"  Removed programs with negative duration")

# Remove programs with extremely long durations (>365 days)
df_filtered = df_filtered[df_filtered['Duration_Days'] <= 365]
print(f"  Removed programs with duration >365 days")

# Remove programs with missing duration data
df_filtered = df_filtered[df_filtered['Duration_Days'].notna()]
print(f"  Removed programs with missing duration")

print(f"\nAfter duration cleanup: {len(df_filtered):,} programs")
print(f"Programs removed in cleanup: {len(df[df['Start_Year'].isin([2023, 2024, 2025]))] - 1:,} programs")

# Show final duration statistics
print(f"\nFinal duration statistics:")
final_duration_stats = df_filtered['Duration_Days'].describe()
print(final_duration_stats)

# Show duration distribution
print(f"\nDuration distribution (ranges):")
duration_ranges = pd.cut(df_filtered['Duration_Days'],
                          bins=[-0.1, 0, 1, 7, 30, 90, 180, 365],
                          labels=['Same day', '1 day', '2-7 days', '1-4 weeks', '1-3 months',
                                  '4-12 months', '13-24 months', '25-36 months', '37-48 months',
                                  '49-60 months', '61-72 months', '73-84 months', '85-96 months',
                                  '97-108 months', '109-120 months', '121-132 months', '133-144 months',
                                  '145-156 months', '157-168 months', '169-180 months', '181-192 months',
                                  '193-204 months', '205-216 months', '217-228 months', '229-240 months',
                                  '241-252 months', '253-264 months', '265-276 months', '277-288 months',
                                  '289-300 months', '301-312 months', '313-324 months', '325-336 months',
                                  '337-348 months', '349-360 months', '361-372 months', '373-384 months',
                                  '385-396 months', '397-408 months', '409-420 months', '421-432 months',
                                  '433-444 months', '445-456 months', '457-468 months', '469-480 months',
                                  '481-492 months', '493-504 months', '505-516 months', '517-528 months',
                                  '529-540 months', '541-552 months', '553-564 months', '565-576 months',
                                  '577-588 months', '589-600 months', '601-612 months', '613-624 months',
                                  '625-636 months', '637-648 months', '649-660 months', '661-672 months',
                                  '673-684 months', '685-696 months', '697-708 months', '709-720 months',
                                  '721-732 months', '733-744 months', '745-756 months', '757-768 months',
                                  '769-780 months', '781-792 months', '793-804 months', '805-816 months',
                                  '817-828 months', '829-840 months', '841-852 months', '853-864 months',
                                  '865-876 months', '877-888 months', '889-900 months', '901-912 months',
                                  '913-924 months', '925-936 months', '937-948 months', '949-960 months',
                                  '961-972 months', '973-984 months', '985-996 months', '997-1008 months',
                                  '1009-1020 months', '1021-1032 months', '1033-1044 months', '1045-1056 months',
                                  '1057-1068 months', '1069-1080 months', '1081-1092 months', '1093-1104 months',
                                  '1105-1116 months', '1117-1128 months', '1129-1140 months', '1141-1152 months',
                                  '1153-1164 months', '1165-1176 months', '1177-1188 months', '1189-1200 months',
                                  '1201-1212 months', '1213-1224 months', '1225-1236 months', '1237-1248 months',
                                  '1249-1260 months', '1261-1272 months', '1273-1284 months', '1285-1296 months',
                                  '1297-1308 months', '1309-1320 months', '1321-1332 months', '1333-1344 months',
                                  '1345-1356 months', '1357-1368 months', '1369-1380 months', '1381-1392 months',
                                  '1393-1404 months', '1405-1416 months', '1417-1428 months', '1429-1440 months',
                                  '1441-1452 months', '1453-1464 months', '1465-1476 months', '1477-1488 months',
                                  '1489-1500 months', '1501-1512 months', '1513-1524 months', '1525-1536 months',
                                  '1537-1548 months', '1549-1560 months', '1561-1572 months', '1573-1584 months',
                                  '1585-1596 months', '1597-1608 months', '1609-1620 months', '1621-1632 months',
                                  '1633-1644 months', '1645-1656 months', '1657-1668 months', '1669-1680 months',
                                  '1681-1692 months', '1693-1704 months', '1705-1716 months', '1717-1728 months',
                                  '1729-1740 months', '1741-1752 months', '1753-1764 months', '1765-1776 months',
                                  '1777-1788 months', '1789-1800 months', '1801-1812 months', '1813-1824 months',
                                  '1825-1836 months', '1837-1848 months', '1849-1860 months', '1861-1872 months',
                                  '1873-1884 months', '1885-1896 months', '1897-1908 months', '1909-1920 months',
                                  '1921-1932 months', '1933-1944 months', '1945-1956 months', '1957-1968 months',
                                  '1969-1980 months', '1981-1992 months', '1993-2004 months', '2005-2016 months',
                                  '2017-2028 months', '2029-2040 months', '2041-2052 months', '2053-2064 months',
                                  '2065-2076 months', '2077-2088 months', '2089-2100 months', '2101-2112 months',
                                  '2113-2124 months', '2125-2136 months', '2137-2148 months', '2149-2160 months',
                                  '2161-2172 months', '2173-2184 months', '2185-2196 months', '2197-2208 months',
                                  '2209-2220 months', '2221-2232 months', '2233-2244 months', '2245-2256 months',
                                  '2257-2268 months', '2269-2280 months', '2281-2292 months', '2293-2304 months',
                                  '2305-2316 months', '2317-2328 months', '2329-2340 months', '2341-2352 months',
                                  '2353-2364 months', '2365-2376 months', '2377-2388 months', '2389-2400 months',
                                  '2401-2412 months', '2413-2424 months', '2425-2436 months', '2437-2448 months',
                                  '2449-2460 months', '2461-2472 months', '2473-2484 months', '2485-2496 months',
                                  '2497-2508 months', '2509-2520 months', '2521-2532 months', '2533-2544 months',
                                  '2545-2556 months', '2557-2568 months', '2569-2580 months', '2581-2592 months',
                                  '2593-2604 months', '2605-2616 months', '2617-2628 months', '2629-2640 months',
                                  '2641-2652 months', '2653-2664 months', '2665-2676 months', '2677-2688 months',
                                  '2689-2700 months', '2701-2712 months', '2713-2724 months', '2725-2736 months',
                                  '2737-2748 months', '2749-2760 months', '2761-2772 months', '2773-2784 months',
                                  '2785-2796 months', '2797-2808 months', '2809-2820 months', '2821-2832 months',
                                  '2833-2844 months', '2845-2856 months', '2857-2868 months', '2869-2880 months',
                                  '2881-2892 months', '2893-2904 months', '2905-2916 months', '2917-2928 months',
                                  '2929-2940 months', '2941-2952 months', '2953-2964 months', '2965-2976 months',
                                  '2977-2988 months', '2989-3000 months', '3001-3012 months', '3013-3024 months',
                                  '3025-3036 months', '3037-3048 months', '3049-3060 months', '3061-3072 months',
                                  '3073-3084 months', '3085-3096 months', '3097-3108 months', '3109-3120 months',
                                  '3121-3132 months', '3133-3144 months', '3145-3156 months', '3157-3168 months',
                                  '3169-3180 months', '3181-3192 months', '3193-3204 months', '3205-3216 months',
                                  '3217-3228 months', '3229-3240 months', '3241-3252 months', '3253-3264 months',
                                  '3265-3276 months', '3277-3288 months', '3289-3300 months', '3301-3312 months',
                                  '3313-3324 months', '3325-3336 months', '3337-3348 months', '3349-3360 months',
                                  '3361-3372 months', '3373-3384 months', '3385-3396 months', '3397-3408 months',
                                  '3409-3420 months', '3421-3432 months', '3433-3444 months', '3445-3456 months',
                                  '3457-3468 months', '3469-3480 months', '3481-3492 months', '3493-3504 months',
                                  '3505-3516 months', '3517-3528 months', '3529-3540 months', '3541-3552 months',
                                  '3553-3564 months', '3565-3576 months', '3577-3588 months', '3589-3600 months',
                                  '3601-3612 months', '3613-3624 months', '3625-3636 months', '3637-3648 months',
                                  '3649-3660 months', '3661-3672 months', '3673-3684 months', '3685-3696 months',
                                  '3697-3708 months', '3709-3720 months', '3721-3732 months', '3733-3744 months',
                                  '3745-3756 months', '3757-3768 months', '3769-3780 months', '3781-3792 months',
                                  '3793-3804 months', '3805-3816 months', '3817-3828 months', '3829-3840 months',
                                  '3841-3852 months', '3853-3864 months', '3865-3876 months', '3877-3888 months',
                                  '3889-3900 months', '3901-3912 months', '3913-3924 months', '3925-3936 months',
                                  '3937-3948 months', '3949-3960 months', '3961-3972 months', '3973-3984 months',
                                  '3985-3996 months', '3997-4008 months', '4009-4020 months', '4021-4032 months',
                                  '4033-4044 months', '4045-4056 months', '4057-4068 months', '4069-4080 months',
                                  '4081-4092 months', '4093-4104 months', '4105-4116 months', '4117-4128 months',
                                  '4129-4140 months', '4141-4152 months', '4153-4164 months', '4165-4176 months',
                                  '4177-4188 months', '4189-4200 months', '4201-4212 months', '4213-4224 months',
                                  '4225-4236 months', '4237-4248 months', '4249-4260 months', '4261-4272 months',
                                  '4273-4284 months', '4285-4296 months', '4297-4308 months', '4309-4320 months',
                                  '4321-4332 months', '4333-4344 months', '4345-4356 months', '4357-4368 months',
                                  '4369-4380 months', '4381-4392 months', '4393-4404 months', '4405-4416 months',
                                  '4417-4428 months', '4429-4440 months', '4441-4452 months', '4453-4464 months',
                                  '4465-4476 months', '4477-4488 months', '4489-4500 months', '4501-4512 months',
                                  '4513-4524 months', '4525-4536 months', '4537-4548 months', '4549-4560 months',
                                  '4561-4572 months', '4573-4584 months', '4585-4596 months', '4597-4608 months',
                                  '4609-4620 months', '4621-4632 months', '4633-4644 months', '4645-4656 months',
                                  '4657-4668 months', '4669-4680 months', '4681-4692 months', '4693-4704 months',
                                  '4705-4716 months', '4717-4728 months', '4729-4740 months', '4741-4752 months',
                                  '4753-4764 months', '4765-4776 months', '4777-4788 months', '4789-4800 months',
                                  '4801-4812 months', '4813-4824 months', '4825-4836 months', '4837-4848 months',
                                  '4849-4860 months', '4861-4872 months', '4873-4884 months', '4885-4896 months',
                                  '4897-4908 months', '4909-4920 months', '4921-4932 months', '4933-4944 months',
                                  '4945-4956 months', '4957-4968 months', '4969-4980 months', '4981-4992 months',
                                  '4993-5004 months', '5005-5016 months', '5017-5028 months', '5029-5040 months',
                                  '5041-5052 months', '5053-5064 months', '5065-5076 months', '5077-5088 months',
                                  '5089-5100 months', '5101-5112 months', '5113-5124 months', '5125-5136 months',
                                  '5137-5148 months', '5149-5160 months', '5161-5172 months', '5173-5184 months',
                                  '5185-5196 months', '5197-5208 months', '5209-5220 months', '5221-5232 months',
                                  '5233-5244 months', '5245-5256 months', '5257-5268 months', '5269-5280 months',
                                  '5281-5292 months', '5293-5304 months', '5305-5316 months', '5317-5328 months',
                                  '5329-5340 months', '5341-5352 months', '5353-5364 months', '5365-5376 months',
                                  '5377-5388 months', '5389-5400 months', '5401-5412 months', '5413-5424 months',
                                  '5425-5436 months', '5437-5448 months', '5449-5460 months', '5461-5472 months',
                                  '5473-5484 months', '5485-5496 months', '5497-5508 months', '5509-5520 months',
                                  '5521-5532 months', '5533-5544 months', '5545-5556 months', '5557-5568 months',
                                  '5569-5580 months', '5581-5592 months', '5593-5604 months', '5605-5616 months',
                                  '5617-5628 months', '5629-5640 months', '5641-5652 months', '5653-5664 months',
                                  '5665-5676 months', '5677-5688 months', '5689-5700 months', '5701-5712 months',
                                  '5713-5724 months', '5725-5736 months', '5737-5748 months', '5749-5760 months',
                                  '5761-5772 months', '5773-5784 months', '5785-5796 months', '5797-5808 months',
                                  '5809-5820 months', '5821-5832 months', '5833-5844 months', '5845-5856 months',
                                  '5857-5868 months', '5869-5880 months', '5881-5892 months', '5893-5904 months',
                                  '5905-5916 months', '5917-5928 months', '5929-5940 months', '5941-5952 months',
                                  '5953-5964 months', '5965-5976 months', '5977-5988 months', '5989-6000 months',
                                  '6001-6012 months', '6013-6024 months', '6025-6036 months', '6037-6048 months',
                                  '6049-6060 months', '6061-6072 months', '6073-6084 months', '6085-6096 months',
                                  '6097-6108 months', '6109-6120 months', '6121-6132 months', '6133-6144 months',
                                  '6145-6156 months', '6157-6168 months', '6169-6180 months', '6181-6192 months',
                                  '6193-6204 months', '6205-6216 months', '6217-6228 months', '6229-6240 months',
                                  '6241-6252 months', '6253-6264 months', '6265-6276 months', '6277-6288 months',
                                  '6289-6300 months', '6301-6312 months', '6313-6324 months', '6325-6336 months',
                                  '6337-6348 months', '6349-6360 months', '6361-6372 months', '6373-6384 months',
                                  '6385-6396 months', '6397-6408 months', '6409-6420 months', '6421-6432 months',
                                  '6433-6444 months', '6445-6456 months', '6457-6468 months', '6469-6480 months',
                                  '6481-6492 months', '6493-6504 months', '6505-6516 months', '6517-6528 months',
                                  '6529-6540 months', '6541-6552 months', '6553-6564 months', '6565-6576 months',
                                  '6577-6588 months', '6589-6600 months', '6601-6612 months', '6613-6624 months',
                                  '6625-6636 months', '6637-6648 months', '6649-6660 months', '6661-6672 months',
                                  '6673-6684 months', '6685-6696 months', '6697-6708 months', '6709-6720 months',
                                  '6721-6732 months', '6733-6744 months', '6745-6756 months', '6757-6768 months',
                                  '6769-6780 months', '6781-6792 months', '6793-6804 months', '6805-6816 months',
                                  '6817-6828 months', '6829-6840 months', '6841-6852 months', '6853-6864 months',
                                  '6865-6876 months', '6877-6888 months', '6889-6900 months', '6901-6912 months',
                                  '6913-6924 months', '6925-6936 months', '6937-6948 months', '6949-6960 months',
                                  '6961-6972 months', '6973-6984 months', '6985-6996 months', '6997-7008 months',
                                  '7009-7020 months', '7021-7032 months', '7033-7044 months', '7045-7056 months',
                                  '7057-7068 months', '7069-7080 months', '7081-7092 months', '7093-7104 months',
                                  '7105-7116 months', '7117-7128 months', '7129-7140 months', '7141-7152 months',
                                  '7153-7164 months', '7165-7176 months', '7177-7188 months', '7189-7200 months',
                                  '7201-7212 months', '7213-7224 months', '7225-7236 months', '7237-7248 months',
                                  '7249-7260 months', '7261-7272 months', '7273-7284 months', '7285-7296 months',
                                  '7297-7308 months', '7309-7320 months', '7321-7332 months', '7333-7344 months',
                                  '7345-7356 months', '7357-7368 months', '7369-7380 months', '7381-7392 months',
                                  '7393-7404 months', '7405-7416 months', '7417-7428 months', '7429-7440 months',
                                  '7441-7452 months', '7453-7464 months', '7465-7476 months', '7477-7488 months',
                                  '7489-7500 months', '7501-7512 months', '7513-7524 months', '7525-7536 months',
                                  '7537-7548 months', '7549-7560 months', '7561-7572 months', '7573-7584 months',
                                  '7585-7596 months', '7597-7608 months', '7609-7620 months', '7621-7632 months',
                                  '7633-7644 months', '7645-7656 months', '7657-7668 months', '7669-7680 months',
                                  '7681-7692 months', '7693-7704 months', '7705-7716 months', '7717-7728 months',
                                  '7729-7740 months', '7741-7752 months', '7753-7764 months', '7765-7776 months',
                                  '7777-7788 months', '7789-7800 months', '7801-7812 months', '7813-7824 months',
                                  '7825-7836 months', '7837-7848 months', '7849-7860 months', '7861-7872 months',
                                  '7873-7884 months', '7885-7896 months', '7897-7908 months', '7909-7920 months',
                                  '7921-7932 months', '7933-7944 months', '7945-7956 months', '7957-7968 months',
                                  '7969-7980 months', '7981-7992 months', '7993-8004 months', '8005-8016 months',
                                  '8017-8028 months', '8029-8040 months', '8041-8052 months', '8053-8064 months',
                                  '8065-8076 months', '8077-8088 months', '8089-8100 months', '8101-8112 months',
                                  '8113-8124 months', '8125-8136 months', '8137-8148 months', '8149-8160 months',
                                  '8161-8172 months', '8173-8184 months', '8185-8196 months', '8197-8208 months',
                                  '8209-8220 months', '8221-8232 months', '8233-8244 months', '8245-8256 months',
                                  '8257-8268 months', '8269-8280 months', '8281-8292 months', '8293-8304 months',
                                  '8305-8316 months', '8317-8328 months', '8329-8340 months', '8341-8352 months',
                                  '8353-8364 months', '8365-8376 months', '8377-8388 months', '8389-8400 months',
                                  '8401-8412 months', '8413-8424 months', '8425-8436 months', '8437-8448 months',
                                  '8449-8460 months', '8461-8472 months', '8473-8484 months', '8485-8496 months',
                                  '8497-8508 months', '8509-8520 months', '8521-8532 months', '8533-8544 months',
                                  '8545-8556 months', '8557-8568 months', '8569-8580 months', '8581-8592 months',
                                  '8593-8604 months', '8605-8616 months', '8617-8628 months', '8629-8640 months',
                                  '8641-8652 months', '8653-8664 months', '8665-8676 months', '8677-8688 months',
                                  '8689-8700 months', '8701-8712 months', '8713-8724 months', '8725-8736 months',
                                  '8737-8748 months', '8749-8760 months', '8761-8772 months', '8773-8784 months',
                                  '8785-8796 months', '8797-8808 months', '8809-8820 months', '8821-8832 months',
                                  '8833-8844 months', '8845-8856 months', '8857-8868 months', '8869-8880 months',
                                  '8881-8892 months', '8893-8904 months', '8905-8916 months', '8917-8928 months',
                                  '8929-8940 months', '8941-8952 months', '8953-8964 months', '8965-8976 months',
                                  '8977-8988 months', '8989-9000 months', '9001-9012 months', '9013-9024 months',
                                  '9025-9036 months', '9037-9048 months', '9049-9060 months', '9061-9072 months',
                                  '9073-9084 months', '9085-9096 months', '9097-9108 months', '9109-9120 months',
                                  '9121-9132 months', '9133-9144 months', '9145-9156 months', '9157-9168 months',
                                  '9169-9180 months', '9181-9192 months', '9193-9204 months', '9205-9216 months',
                                  '9217-9228 months', '9229-9240 months', '9241-9252 months', '9253-9264 months',
                                  '9265-9276 months', '9277-9288 months', '9289-9300 months', '9301-9312 months',
                                  '9313-9324 months', '9325-9336 months', '9337-9348 months', '9349-9360 months',
                                  '9361-9372 months', '9373-9384 months', '9385-9396 months', '9397-9408 months',
                                  '9409-9420 months', '9421-9432 months', '9433-9444 months', '9445-9456 months',
                                  '9457-9468 months', '9469-9480 months', '9481-9492 months', '9493-9504 months',
                                  '9505-9516 months', '9517-9528 months', '9529-9540 months', '9541-9552 months',
                                  '9553-9564 months', '9565-9576 months', '9577-9588 months', '9589-9600 months',
                                  '9601-9612 months', '9613-9624 months', '9625-9636 months', '9637-9648 months',
                                  '9649-9660 months', '9661-9672 months', '9673-9684 months', '9685-9696 months',
                                  '9697-9708 months', '9709-9720 months', '9721-9732 months', '9733-9744 months',
                                  '9745-9756 months', '9757-9768 months', '9769-9780 months', '9781-9792 months',
                                  '9793-9804 months', '9805-9816 months', '9817-9828 months', '9829-9840 months',
                                  '9841-9852 months', '9853-9864 months', '9865-9876 months', '9877-9888 months',
                                  '9889-9900 months', '9901-9912 months', '9913-9924 months', '9925-9936 months',
                                  '9937-9948 months', '9949-9960 months', '9961-9972 months', '9973-9984 months',
                                  '9985-9996 months', '9997-10008 months', '10009-10020 months', '10021-10032 months',
                                  '10033-10044 months', '10045-10056 months', '10057-10068 months', '10069-10080 months',
                                  '10081-10092 months', '10093-10104 months', '10105-10116 months', '10117-10128 months',
                                  '10129-10140 months', '10141-10152 months', '10153-10164 months', '10165-10176 months',
                                  '10177-10188 months', '10189-10200 months', '10201-10212 months', '10213-10224 months',
                                  '10225-10236 months', '10237-10248 months', '10249-10260 months', '10261-10272 months',
                                  '10273-10284 months', '10285-10296 months', '10297-10308 months', '10309-10320 months',
                                  '10321-10332 months', '10333-10344 months', '10345-10356 months', '10357-10368 months',
                                  '10369-10380 months', '10381-10392 months', '10393-10404 months', '10405-10416 months',
                                  '10417-10428 months', '10429-10440 months', '10441-10452 months', '10453-10464 months',
                                  '10465-10476 months', '10477-10488 months', '10489-10500 months', '10501-10512 months',
                                  '10513-10524 months', '10525-10536 months', '10537-10548 months', '10549-10560 months',
                                  '10561-10572 months', '10573-10584 months', '10585-10596 months', '10597-10608 months',
                                  '10609-10620 months', '10621-10632 months', '10633-10644 months', '10645-10656 months',
                                  '10657-10668 months', '10669-10680 months', '10681-10692 months', '10693-10704 months',
                                  '10705-10716 months', '10717-10728 months', '10729-10740 months', '10741-10752 months',
                                  '10753-10764 months', '10765-10776 months', '10777-10788 months', '10789-10800 months',
                                  '10801-10812 months', '10813-10824 months', '10825-10836 months', '10837-10848 months',
                                  '10849-10860 months', '10861-10872 months', '10873-10884 months', '10885-10896 months',
                                  '10897-10908 months', '10909-10920 months', '10921-10932 months', '10933-10944 months',
                                  '10945-10956 months', '10957-10968 months', '10969-10980 months', '10981-10992 months',
                                  '10993-11004 months', '11005-11016 months', '11017-11028 months', '11029-11040 months',
                                  '11041-11052 months', '11053-11064 months', '11065-11076 months', '11077-11088 months',
                                  '11089-11100 months', '11101-11112 months', '11113-11124 months', '11125-11136 months',
                                  '11137-11148 months', '11149-11160 months', '11161-11172 months', '11173-11184 months',
                                  '11185-11196 months', '11197-11208 months', '11209-11220 months', '11221-11232 months',
                                  '11233-11244 months', '11245-11256 months', '11257-11268 months', '11269-11280 months',
                                  '11281-11292 months', '11293-11304 months', '11305-11316 months', '11317-11328 months',
                                  '11329-11340 months',
```

```
for range_label, count in duration_counts.items():
    percentage = (count / len(df_filtered)) * 100
    print(f" {range_label}: {count:,} programs ({percentage:.1f}%)" )
```

DURATION-BASED DATA CLEANUP

=====

Before duration cleanup: 319,987 programs

Duration quality issues in filtered data:

- Negative duration: 0 programs
- Very long duration (>365 days): 616 programs
- Zero duration (same day programs): 179,038 programs
- Missing duration: 0 programs

Applying duration filters...

- Removed programs with negative duration
- Removed programs with duration >365 days
- Removed programs with missing duration

After duration cleanup: 319,371 programs

Programs removed in cleanup: 616

Final duration statistics:

```
count      319371.000000
mean         27.499645
std         36.845430
min          0.000000
25%          0.000000
50%          0.000000
75%         63.000000
max         365.000000
Name: Duration_Days, dtype: float64
```

Duration distribution (ranges):

- Same day: 179,038 programs (56.1%)
- 1 day: 167 programs (0.1%)
- 2-7 days: 3,597 programs (1.1%)
- 1-4 weeks: 6,615 programs (2.1%)
- 1-3 months: 115,562 programs (36.2%)
- 3-6 months: 13,015 programs (4.1%)
- 6-12 months: 1,377 programs (0.4%)

Why these duration filters

- Negative durations usually mean swapped start/end dates.
- 365-day spans look like data entry errors for youth programs.
- Dropping NAs keeps later grouping math simple. If your use-case needs camps that run all year, loosen these bounds here.

Bucket program durations

We convert raw `Duration_Days` into a small set of interpretable bins (same day, 1 day, 2-7 days, etc.). These buckets make it easier to segment programs and later join duration as a categorical feature.

```
# Create Duration_Group column as a separate feature for analysis
print("CREATING DURATION_GROUP COLUMN")
print("="*35)

# Create categorical column based on duration ranges
duration_ranges = pd.cut(df_filtered['Duration_Days'],
                        bins=[-0.1, 0, 1, 7, 30, 90, 180, 365],
                        labels=['Same day', '1 day', '2-7 days', '1-4 weeks', '1-3 months',
                              '4-12 months', '13-365 months'],
                        include_lowest=True)

df_filtered['Duration_Group'] = duration_ranges
print(f" Created 'Duration_Group' column with {df_filtered['Duration_Group'].nunique()} categories")

df_filtered.head()

print(f"\n Duration_Group column ready for further analysis and grouping!")
```

CREATING DURATION_GROUP COLUMN

=====

Created 'Duration_Group' column with 7 categories

Duration_Group column ready for further analysis and grouping!

EDA: Check category balance

Category counts help us spot label imbalance (rare vs. common program types) before we build any models or sampling strategies.

```
# Analyze Category Name distribution in the filtered dataset
print("CATEGORY NAME ANALYSIS")
print("="*30)

# Compute value counts for Category Name
category_counts = df_filtered['Category Name'].value_counts()
print(f"Total unique categories: {df_filtered['Category Name'].nunique()}")
print(f"Total programs: {len(df_filtered):,}")

print(f"\nCategory distribution:")
print(f"Top 20 most common categories:")
for i, (category, count) in enumerate(category_counts.head(20).items(), 1):
    percentage = (count / len(df_filtered)) * 100
    print(f"  {i:2d}. {category}: {count:,} programs ({percentage:.1f}%)")

# Show categories with very few programs
print(f"\nCategories with fewer than 100 programs:")
rare_categories = category_counts[category_counts < 100]
print(f"Number of rare categories: {len(rare_categories)}")
for category, count in rare_categories.items():
    print(f"  - {category}: {count} programs")

# Show summary statistics
print(f"\nCategory statistics:")
print(f"- Most popular category: {category_counts.index[0]} ({category_counts.iloc[0]:,} programs)")
print(f"- Average programs per category: {category_counts.mean():.1f}")
print(f"- Median programs per category: {category_counts.median():.1f}")
print(f"- Categories with >1000 programs: {(category_counts > 1000).sum()}")
print(f"- Categories with >500 programs: {(category_counts > 500).sum()}")
print(f"- Categories with >100 programs: {(category_counts > 100).sum()}")
```

CATEGORY NAME ANALYSIS

=====

Total unique categories: 21

Total programs: 319,371

Category distribution:

Top 20 most common categories:

1. Sports + Wellness.: 122,148 programs (38.2%)
2. Music & Art.: 105,679 programs (33.1%)
3. Reading & Writing.: 34,538 programs (10.8%)
4. Academic Support: 13,007 programs (4.1%)
5. Science: 9,442 programs (3.0%)
6. Computers.: 9,297 programs (2.9%)
7. Building & Fixing Things: 6,617 programs (2.1%)
8. Helping Your Community.: 3,223 programs (1.0%)
9. Performance.: 2,624 programs (0.8%)
10. Nature.: 2,569 programs (0.8%)
11. Managing Money.: 2,177 programs (0.7%)
12. Healthcare: 2,161 programs (0.7%)
13. Food.: 1,767 programs (0.6%)
14. Work + Career: 1,665 programs (0.5%)
15. Digital Media.: 1,557 programs (0.5%)
16. Social Studies: 499 programs (0.2%)
17. Teaching: 167 programs (0.1%)
18. Customer/Human Service: 94 programs (0.0%)
19. Math: 60 programs (0.0%)
20. Law: 54 programs (0.0%)

Categories with fewer than 100 programs:

Number of rare categories: 4

- Customer/Human Service: 94 programs
- Math: 60 programs
- Law: 54 programs
- Transportation: 26 programs

Category statistics:

- Most popular category: Sports + Wellness. (122,148 programs)
- Average programs per category: 15208.1
- Median programs per category: 2177.0
- Categories with >1000 programs: 15
- Categories with >500 programs: 15
- Categories with >100 programs: 17

How to use category balance

- Imbalanced labels influence both clustering visuals and any supervised baselines.
- Consider collapsing very rare categories or reweighting later if you plan to train classifiers.
- Keep a note of the top categories; later notebooks filter to them.

Text Analysis Phase

Now that we have a clean, filtered dataset with 2023-2025 programs, let's dive into the core text analysis. We'll focus on analyzing the **Program Name** and **Description** fields to extract meaningful insights.

Text Preprocessing & Cleaning

- Clean HTML tags and special characters from descriptions
- Standardize text format (lowercase, remove punctuation)
- Remove stop words and short words
- Handle common text issues

Peek at raw text

A quick look at a few names and descriptions grounds the cleaning choices - we want to see real noise like HTML tags, boilerplate, or contact info before writing a cleaning function.

```
# 1. TEXT PREPROCESSING & CLEANING
print("TEXT PREPROCESSING & CLEANING")
print("="*40)

import re
import string
from collections import Counter

# Sample text data to understand what we're working with
print(f"\n Sample Program Names:")
sample_names = df_filtered['Program Name'].dropna().head()
for i, name in enumerate(sample_names, 1):
    print(f"  {i}. {name}")

print(f"\n Sample Descriptions (first 500 chars):")
sample_descriptions = df_filtered['Description'].dropna().head(3)
for i, desc in enumerate(sample_descriptions, 1):
    print(f"  {i}. {str(desc)[:500]}...")

print("\n" + "="*50)
```

TEXT PREPROCESSING & CLEANING

=====

Sample Program Names:

1. Homework Help: Virtual Teacher in the Library
2. Art of Storytelling Learning Circle
3. Learn American Sign Language
4. Learn American Sign Language
5. The Harold Washington Experience: Historians Reflect on His Impact

Sample Descriptions (first 500 chars):

1. Get homework help from a certified teacher and learn about library resources with our ex
2. What makes for a good story? In this course developed by Pixar, learn how a story's stru
3. Want to learn or brush up on your American Sign Language skills? Join the Thurgood Marsh

=====

Build a reusable cleaning function

We normalize text by lowercasing, stripping HTML/URLs/emails/numbers, removing stopwords, and dropping very short tokens. This creates `Program_Name_Clean` and `Description_Clean` fields ready for vectorization.

Setup note for stopwords

`nltk.download('stopwords')` only needs to run once per environment. If you're on a locked-down machine, you can vendor the stopwords list manually instead of downloading at runtime.

```
from nltk.corpus import stopwords
import nltk
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

# Create text cleaning function
def clean_text(text, min_length=3):
    """Clean and normalize text for analysis"""
    if pd.isna(text) or text == '':
        return ''

    # Convert to string and lowercase
    text = str(text).lower()

    # Remove HTML tags (basic cleaning)
```

```

text = re.sub(r'<[^>]+>', ' ', text)

# Remove URLs
text = re.sub(r'http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[*\\(\)\,]|(?:%[0-9a-fA-F][0-9a-

# Remove email addresses
text = re.sub(r'\S+@\S+', ' ', text)

# Remove numbers
text = re.sub(r'[~a-zA-Z\s]', '', text)

# Remove extra whitespace and punctuation
text = re.sub(r'[~\w\s]', ' ', text)
text = re.sub(r'\s+', ' ', text).strip()

# Remove Stopwords - Common words that add little meaning (e.g., 'the', 'is', 'in')
words = [word for word in text.split()
          if len(word) >= min_length and word not in stop_words]

return ' '.join(words)

# Apply text cleaning
print(" Cleaning text fields...")
df_filtered['Program_Name_Clean'] = df_filtered['Program Name'].apply(clean_text)
df_filtered['Description_Clean'] = df_filtered['Description'].apply(clean_text)

# Show before/after examples
print(f"\n Before/After Cleaning Examples:")
for i in range(3):
    if i < len(df_filtered):
        print(f"\nExample {i+1}:")
        original_name = df_filtered.iloc[i]['Program Name']
        clean_name = df_filtered.iloc[i]['Program_Name_Clean']
        print(f"  Original Name: {original_name}")
        print(f"  Clean Name: {clean_name}")

        original_desc = str(df_filtered.iloc[i]['Description'])[:150] + "..."
        clean_desc = df_filtered.iloc[i]['Description_Clean'][:150] + "..."
        print(f"  Original Desc: {original_desc}")
        print(f"  Clean Desc: {clean_desc}")
        print("-" * 60)

```

```
[nltk_data] Downloading package stopwords to
[nltk_data]      /Users/R8C6026/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Cleaning text fields...

Before/After Cleaning Examples:

Example 1:

Original Name: Homework Help: Virtual Teacher in the Library

Clean Name: homework help virtual teacher library

Original Desc: Get homework help from a certified teacher and learn about library resources

Clean Desc: get homework help certified teacher learn library resources expert librarians

Example 2:

Original Name: Art of Storytelling Learning Circle

Clean Name: art storytelling learning circle

Original Desc: What makes for a good story? In this course developed by Pixar, learn how a

Clean Desc: makes good story course developed pixar learn storys structure characters visu

Example 3:

Original Name: Learn American Sign Language

Clean Name: learn american sign language

Original Desc: Want to learn or brush up on your American Sign Language skills? Join the TL

Clean Desc: want learn brush american sign language skills join thurgood marshall branch w

Add simple length features

Word counts for names and descriptions (Name_Word_Count, Desc_Word_Count) give a quick quality check and can serve as lightweight features or filters.

```
# add word count columns for program name and description
s = df_filtered["Program_Name_Clean"].astype("string")
df_filtered["Name_Word_Count"] = s.str.split().str.len()

s = df_filtered["Description_Clean"].astype("string")
df_filtered["Desc_Word_Count"] = s.str.split().str.len()
df_filtered.head()
```

	Program ID	Program Name	Description
6	231612	Homework Help: Virtual Teacher in the Library	Get homework help from a certified t
17	220338	Art of Storytelling Learning Circle	What makes for a good story? In thi
19	156338	Learn American Sign Language	Want to learn or brush up on your A
20	211006	Learn American Sign Language	Want to learn or brush up on your A
22	199268	The Harold Washington Experience: Historians R...	In April 1983, Harold Washington m

Quick exploratory text stats

Basic length stats and top tokens show what dominates the corpus and whether extra cleaning or rebalancing is needed.

```
# 2. EXPLORATORY TEXT ANALYSIS
print("\n" + "="*50)
print("EXPLORATORY TEXT ANALYSIS")
print("="*50)

# Text length analysis
print("\nProgram Names (cleaned) Statistics:")

name_stats = df_filtered['Name_Word_Count'].describe()
print(f"  Mean: {name_stats['mean']:.1f} words")
print(f"  Median: {name_stats['50%']:.1f} words")
print(f"  Range: {name_stats['min']:.0f} - {name_stats['max']:.0f} words")

# Word frequency analysis
print(f"\n Word Frequency Analysis:")

# Most common words in program names
name_words = []
for name in df_filtered['Program_Name_Clean'].dropna():
    name_words.extend(name.split())

name_word_counts = Counter(name_words)
print(f"\nTop 15 words in Program Names:")
for word, count in name_word_counts.most_common(15):
    print(f"  {word}: {count:,} times")

print("\nProgram Description (cleaned) Statistics:")
```

```

desc_stats = df_filtered['Desc_Word_Count'].describe()
print(f" Mean: {desc_stats['mean']:.1f} words")
print(f" Median: {desc_stats['50%']:.1f} words")
print(f" Range: {desc_stats['min']:.0f} - {desc_stats['max']:.0f} words")

# Most common words in descriptions
desc_words = []
for desc in df_filtered['Description_Clean'].dropna():
    desc_words.extend(desc.split())

desc_word_counts = Counter(desc_words)
print(f"\nTop 15 words in Descriptions:")
for word, count in desc_word_counts.most_common(15):
    print(f" {word}: {count:,} times")

# Text quality assessment
print(f"\n Text Quality Assessment based on desc_word_count:")
very_long_descs = (df_filtered['Desc_Word_Count'] > 500).sum()
print(f"Very long descriptions (>500 words): {very_long_descs:,} ({very_long_descs/len(df_filtered['Description_Clean'])*100}% of total)")

print("\n Text preprocessing and exploratory analysis complete!")

```

```

=====
EXPLORATORY TEXT ANALYSIS
=====

```

Program Names (cleaned) Statistics:

```

Mean: 3.8 words
Median: 3.0 words
Range: 0 - 27 words

```

Word Frequency Analysis:

Top 15 words in Program Names:

```

time: 22,623 times
open: 20,845 times
story: 19,232 times
club: 17,661 times
gymnastics: 17,323 times
park: 14,920 times

```

ice: 14,649 times
swim: 12,533 times
mcfetridge: 11,853 times
basketball: 11,419 times
film: 11,377 times
camp: 11,281 times
screening: 11,069 times
teen: 9,893 times
skating: 9,811 times

Program Description (cleaned) Statistics:

Mean: 45.9 words
Median: 38.0 words
Range: 1 - 757 words

Top 15 words in Descriptions:

event: 326,678 times
days: 216,090 times
accessibility: 191,723 times
email: 180,708 times
must: 174,776 times
please: 170,029 times
library: 146,332 times
week: 138,923 times
least: 136,703 times
call: 135,389 times
assistance: 131,891 times
language: 131,135 times
made: 130,858 times
business: 130,181 times
sign: 130,009 times

Text Quality Assessment based on desc_word_count:

Very long descriptions (>500 words): 20 (0.0%)

Text preprocessing and exploratory analysis complete!

Add a Domain-Specific Stoplist

As seen in the results above, program descriptions often contain repeated boilerplate phrases (e.g., “*registration required*”, “*free*”, “*north side*”, or phone numbers).

To reduce noise and improve feature quality, create a custom stoplist called `domain_stop_words` and apply it to further clean the descriptions.

```
# add domain stop words
domain_stop_words = {'event', 'attend', 'attending', 'need', 'using', 'fun', 'email', 'must',
                    'contact', 'information', 'phone', 'required', 'register', 'registration', 'free',
                    'learn', 'session', 'provide', 'available', 'time', 'date', 'location', 'virtual', 'in-person', 'months'}

# Further clean the Description_Clean column by removing domain-specific stop words, months
def remove_domain_stopwords(text):
    if pd.isna(text) or text == '':
        return ''
    words = text.split()
    filtered_words = [word for word in words if word not in domain_stop_words]
    return ' '.join(filtered_words)
df_filtered['Description_Clean'] = df_filtered['Description_Clean'].apply(remove_domain_stopwords)
print(f" Removed domain-specific stop words from 'Description_Clean' column")

# Most common words in descriptions
desc_words = []
for desc in df_filtered['Description_Clean'].dropna():
    desc_words.extend(desc.split())

desc_word_counts = Counter(desc_words)
print(f"\nTop 15 words in Descriptions:")
for word, count in desc_word_counts.most_common(15):
    print(f" {word}: {count:,} times")
```

Removed domain-specific stop words from 'Description_Clean' column

Top 15 words in Descriptions:

```
library: 146,332 times
interpretation: 127,900 times
accommodations: 127,305 times
play: 78,704 times
check: 69,037 times
cpl: 66,110 times
place: 60,163 times
encouraged: 57,749 times
takes: 57,578 times
person: 56,296 times
games: 53,269 times
```

```
level: 53,265 times
park: 51,211 times
visiting: 50,847 times
strongly: 49,549 times
```

Tuning the domain stoplist

Iterate on this list as you explore: copy/paste high-frequency boilerplate phrases you still see in top tokens or word clouds. Save a dated version so teammates know which run produced which cleaned file.

```
df_filtered.groupby('Category Name').size()
```

Category Name	
Academic Support	13007
Building & Fixing Things	6617
Computers.	9297
Customer/Human Service	94
Digital Media.	1557
Food.	1767
Healthcare	2161
Helping Your Community.	3223
Law	54
Managing Money.	2177
Math	60
Music & Art.	105679
Nature.	2569
Performance.	2624
Reading & Writing.	34538
Science	9442
Social Studies	499
Sports + Wellness.	122148
Teaching	167
Transportation	26
Work + Career	1665

dtype: int64

Save the cleaned dataset

We export the filtered, cleaned records to `datasets/data_cleaned_20250826.csv` so downstream notebooks can focus on modeling without repeating the prep steps.

```
df_filtered.to_csv('datasets/data_cleaned_20250826.csv', index=False)
```

```
df_filtered.shape, df.shape, len(df_filtered)/len(df)
```

```
((319371, 15), (460531, 10), 0.6934842605600926)
```

```
df_filtered['Category Name'].value_counts().index.to_list()
```

```
['Sports + Wellness.',  
 'Music & Art.',  
 'Reading & Writing.',  
 'Academic Support',  
 'Science',  
 'Computers.',  
 'Building & Fixing Things',  
 'Helping Your Community.',  
 'Performance.',  
 'Nature.',  
 'Managing Money.',  
 'Healthcare',  
 'Food.',  
 'Work + Career',  
 'Digital Media.',  
 'Social Studies',  
 'Teaching',  
 'Customer/Human Service',  
 'Math',  
 'Law',  
 'Transportation']
```

```
df_filtered.sample(1).to_dict()
```

```
{'Program ID': {59769: '241497'},  
 'Program Name': {59769: 'Flag Football at Addams'},  
 'Description': {59769: 'The proper fundamentals and techniques of football will be taught in'},  
 'Category Name': {59769: 'Sports + Wellness.'},  
 'Start Date': {59769: Timestamp('2025-03-31 00:00:00')},  
 'End Date': {59769: Timestamp('2025-05-19 00:00:00')},  
 'Start Time': {59769: '18:00'},
```

```
'End Time': {59769: '18:45'},
'Start_Year': {59769: 2025},
'Duration_Days': {59769: 49.0},
'Duration_Group': {59769: '1-3 months'},
'Program_Name_Clean': {59769: 'flag football addams'},
'Description_Clean': {59769: 'proper fundamentals techniques football taught players receive'},
'Name_Word_Count': {59769: 3},
'Desc_Word_Count': {59769: 24}}
```

Wrap-up and next steps

You now have a cleaned, filtered, and tokenized dataset saved to `datasets/data_cleaned_20250826.csv`.
- Hand off to clustering/classification notebooks (`2_text_analysis_category_clustering.ipynb`, `3_text_analysis_category_quality.ipynb`). - If results look off later, revisit the checkpoints above (year filter size, duration thresholds, stoplist contents) and adjust with notes so analyses stay reproducible.